

Developing Apps on QT **Part—5**

In the previous articles, we learned how to use the graphical widgets. Now let us go ahead and work on making your graphical application look even better.

We can place the widget in the window without having to bother about the arrangement. Qt provides us with the mechanism to place the widgets with respect to the x and y co-ordinates of the parent window. This helps us to make the application look better.

Manual layout

In the manual layout, we need to specify the x and y coordinates of the child widget, as shown in this example below:

```
mywidget::mywidget(QWidget *parent) :
    QWidget(parent)
{
    setFixedSize(400, 75);
    label1 = new QLabel(this);
    label1->setText("Name");
    label1->setGeometry(10,10,150,25);

    line1 = new QLineEdit(this);
    line1->setGeometry(200,10,150,25);
}
```

The class *mywidget* has been inherited from the *QWidget* class. The public function *void setFixedSize(int height, int width)* of the *QWidget* class has been used to set the size of the *mywidget* class object. The *Qsize* class provides the size of a 2-D object. The size is specified by the *width()* and *height()*. The size of the *mywidget* size can also be set by the following code:

```
QSize size;
size.setHeight(75);
size.setWidth(400);
setFixedSize(size);
```

There are two more functions available to fix the size of a widget. The *setFixedHeight(int)* function will set the height and the width can be set by the *setFixedWidth(int)* member function.

There are also some options to set the minimum and maximum size. The minimum size can be specified by *setMinimumSize(int, int)* and maximum by *setMaximumSize(int, int)*. Both these options can be used when you want your application window to occupy the appropriate space.

Laying out the child widgets

The next task is to lay out the child widgets in the parent. Go through the following sample code:

```
label1 = new QLabel(this);
label1->setText("Name");
label1->setGeometry(10,10,150,25);
line1 = new QLineEdit(this);
line1->setGeometry(200,10,150,25);
```

Here we have created a *label* and a *lineedit*. To place these widgets properly in the parent, we need to have two things – the first is the location (x,y co-ordinates) and the second is the size. The function *setGeometry(int x, int y, int width, int height)* is sufficient to specify these things. In this way, you can place the child widgets according to your requirements.

Automatic layout

We have just seen how to define the size and create a manual layout. Now we can create any type of layout using this method. But like every manual task, this also consumes a lot of time. So, there ought to be some mechanism that can manage the size and the co-ordinates of the widgets by itself. The automatic layout mechanism of Qt is the solution to the problem. It provides pre-defined layouts in which we can add our widgets. The basic automatic layouts are horizontal and vertical.

They can be used either directly from the design view or from the code. Let us try using it by changing the code, because though the designer method is simple and fast, the coding option leads to better understanding.

BoxLayout

The horizontal layout grows towards the right and the vertical layout grows towards the bottom. *QHBoxLayout* and *QVBoxLayout* are provided by Qt for this purpose. Both these classes are inherited from the *QBoxLayout* class. Now let us create a *BoxLayout* and then add the widgets to it. The code given below is also available at http://www.linuxforu.com/article_source_code/may12/qt-part5.zip

```
#include <QtGui/QApplication>
#include <QtGui>
#include <QtCore>
int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    QWidget widget;
    QLabel *label;
    QLineEdit *line;
    QHBoxLayout *hbox;
```